

```
from google.colab import files
uploaded = files.upload()
```

Choose Files HospitalOP..._10000.csv

HospitalOPD_BottleneckLabelled_10000.csv(text/csv) - 3199093 bytes, last modified: 9/23/2026 - 100% done
Saving HospitalOPD_BottleneckLabelled_10000.csv to HospitalOPD_BottleneckLabelled_10000 (1).csv

```
import pandas as pd

df = pd.read_csv("HospitalOPD_BottleneckLabelled_10000.csv", encoding='latin1')
df.head()
```

	appointment_id	patient_id	event_id	event_date	appointment_date_and_time	prescription_doctor_start_date_and_time	prescri
0	APT00001	PAT00001	EVT00001	3/23/2026	3/23/2026 8:57	3/23/2026 9:30	
1	APT00002	PAT00002	EVT00002	2/5/2026	2/5/2026 9:19	2/5/2026 9:35	
2	APT00003	PAT00003	EVT00003	2/18/2026	2/18/2026 10:22	2/18/2026 11:14	
3	APT00004	PAT00004	EVT00004	1/28/2026	1/28/2026 13:35	1/28/2026 13:58	
4	APT00005	PAT00005	EVT00005	4/21/2026	4/21/2026 8:05	4/21/2026 8:24	

5 rows × 46 columns

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 46 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   appointment_id                           10000 non-null  object
1   patient_id                               10000 non-null  object
2   event_id                                 10000 non-null  object
3   event_date                               10000 non-null  object
4   appointment_date_and_time                10000 non-null  object
5   prescription_doctor_start_date_and_time  10000 non-null  object
6   prescription_doctor_end_date_and_time    10000 non-null  object
7   lab_start_date_and_time                  5041 non-null   object
8   lab_completed_date_and_time              5041 non-null   object
9   lab_report_generation_date_and_time      5041 non-null   object
10  image_start_date_and_time                4986 non-null   object
11  image_end_date_and_time                  4986 non-null   object
12  image_report_generation_date_and_time    4986 non-null   object
13  has_appointment                          10000 non-null  int64
14  has_prescription                         10000 non-null  int64
15  has_lab                                  10000 non-null  int64
16  has_imaging                             10000 non-null  int64
17  doctor_id                               10000 non-null  object
18  doctor_name                             10000 non-null  object
19  case_id                                  10000 non-null  object
20  activity                                 10000 non-null  object
21  timestamp                                10000 non-null  object
22  event_order                             10000 non-null  int64
23  source_activity                         10000 non-null  object
24  target_activity                         10000 non-null  object
25  transition_frequency                    10000 non-null  int64
26  transition_probability                  10000 non-null  float64
27  waiting_time_minutes                    10000 non-null  int64
28  processing_duration_minutes              10000 non-null  int64
29  throughput_time_minutes                  10000 non-null  int64
30  transition_delay_minutes                 10000 non-null  int64
31  cumulative_delay_minutes                 10000 non-null  int64
32  concurrent_case_load                     10000 non-null  int64
33  activity_frequency                       10000 non-null  int64
34  repeated_activity_count                  10000 non-null  int64
35  trace_length                            10000 non-null  int64
36  variant_id                              10000 non-null  object
37  available_human_resources                 10000 non-null  int64
38  activity_resource_load                     10000 non-null  float64
39  doctor_case_load                         10000 non-null  int64
40  degree_centrality                       10000 non-null  float64
41  betweenness_centrality                   10000 non-null  float64
42  closeness_centrality                     10000 non-null  float64
43  eigenvector_centrality                   10000 non-null  float64
44  eccentricity_centrality                   10000 non-null  float64
45  bottleneck_label                         10000 non-null  int64
dtypes: float64(7), int64(18), object(21)
memory usage: 3.5+ MB
```

	has_appointment	has_prescription	has_lab	has_imaging	event_order	transition_frequency	transition_probability
count	10000.0	10000.0	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	1.0	1.0	0.504100	0.498600	5.520700	529.699500	0.551096
std	0.0	0.0	0.500008	0.500023	2.877116	275.304566	0.259682
min	1.0	1.0	0.000000	0.000000	1.000000	50.000000	0.100000
25%	1.0	1.0	0.000000	0.000000	3.000000	292.000000	0.330000
50%	1.0	1.0	1.000000	0.000000	6.000000	527.500000	0.550000
75%	1.0	1.0	1.000000	1.000000	8.000000	769.000000	0.770000
max	1.0	1.0	1.000000	1.000000	10.000000	1000.000000	1.000000

8 rows × 25 columns

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 46 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   appointment_id                           10000 non-null  object
1   patient_id                               10000 non-null  object
```

```

2  event_id                10000 non-null object
3  event_date              10000 non-null object
4  appointment_date_and_time 10000 non-null object
5  prescription_doctor_start_date_and_time 10000 non-null object
6  prescription_doctor_end_date_and_time 10000 non-null object
7  lab_start_date_and_time  5041 non-null object
8  lab_completed_date_and_time 5041 non-null object
9  lab_report_generation_date_and_time 5041 non-null object
10 image_start_date_and_time 4986 non-null object
11 image_end_date_and_time  4986 non-null object
12 image_report_generation_date_and_time 4986 non-null object
13 has_appointment          10000 non-null int64
14 has_prescription          10000 non-null int64
15 has_lab                   10000 non-null int64
16 has_imaging               10000 non-null int64
17 doctor_id                 10000 non-null object
18 doctor_name               10000 non-null object
19 case_id                   10000 non-null object
20 activity                  10000 non-null object
21 timestamp                 10000 non-null object
22 event_order               10000 non-null int64
23 source_activity           10000 non-null object
24 target_activity           10000 non-null object
25 transition_frequency       10000 non-null int64
26 transition_probability     10000 non-null float64
27 waiting_time_minutes       10000 non-null int64
28 processing_duration_minutes 10000 non-null int64
29 throughput_time_minutes    10000 non-null int64
30 transition_delay_minutes    10000 non-null int64
31 cumulative_delay_minutes    10000 non-null int64
32 concurrent_case_load       10000 non-null int64
33 activity_frequency          10000 non-null int64
34 repeated_activity_count     10000 non-null int64
35 trace_length               10000 non-null int64
36 variant_id                 10000 non-null object
37 available_human_resources   10000 non-null int64
38 activity_resource_load      10000 non-null float64
39 doctor_case_load            10000 non-null int64
40 degree_centrality           10000 non-null float64
41 betweenness_centrality      10000 non-null float64
42 closeness_centrality        10000 non-null float64
43 eigenvector_centrality      10000 non-null float64
44 eccentricity_centrality      10000 non-null float64
45 bottleneck_label           10000 non-null int64

```

dtypes: float64(7), int64(18), object(21)

memory usage: 3.5+ MB

df.head()

	appointment_id	patient_id	event_id	event_date	appointment_date_and_time	prescription_doctor_start_date_and_time	prescription_doctor_end_date_and_time
0	APT00001	PAT00001	EVT00001	3/23/2026	3/23/2026 8:57	3/23/2026 9:30	
1	APT00002	PAT00002	EVT00002	2/5/2026	2/5/2026 9:19	2/5/2026 9:35	
2	APT00003	PAT00003	EVT00003	2/18/2026	2/18/2026 10:22	2/18/2026 11:14	
3	APT00004	PAT00004	EVT00004	1/28/2026	1/28/2026 13:35	1/28/2026 13:58	
4	APT00005	PAT00005	EVT00005	4/21/2026	4/21/2026 8:05	4/21/2026 8:24	

5 rows × 46 columns

```

import pandas as pd
import matplotlib.pyplot as plt

# تحويل التواريخ
time_cols = [
    'event_date', 'appointment_date_and_time',
    'prescription_doctor_start_date_and_time', 'prescription_doctor_end_date_and_time',
    'lab_start_date_and_time', 'lab_completed_date_and_time', 'lab_report_generation_date_and_time',
    'image_start_date_and_time', 'image_end_date_and_time', 'image_report_generation_date_and_time',
    'timestamp'
]

for col in time_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce')

```

```

activity_bottleneck = (
    df.groupby('activity')['bottleneck_label']
    .mean()
    .sort_values(ascending=False) * 100
)

activity_bottleneck.head(10)

```

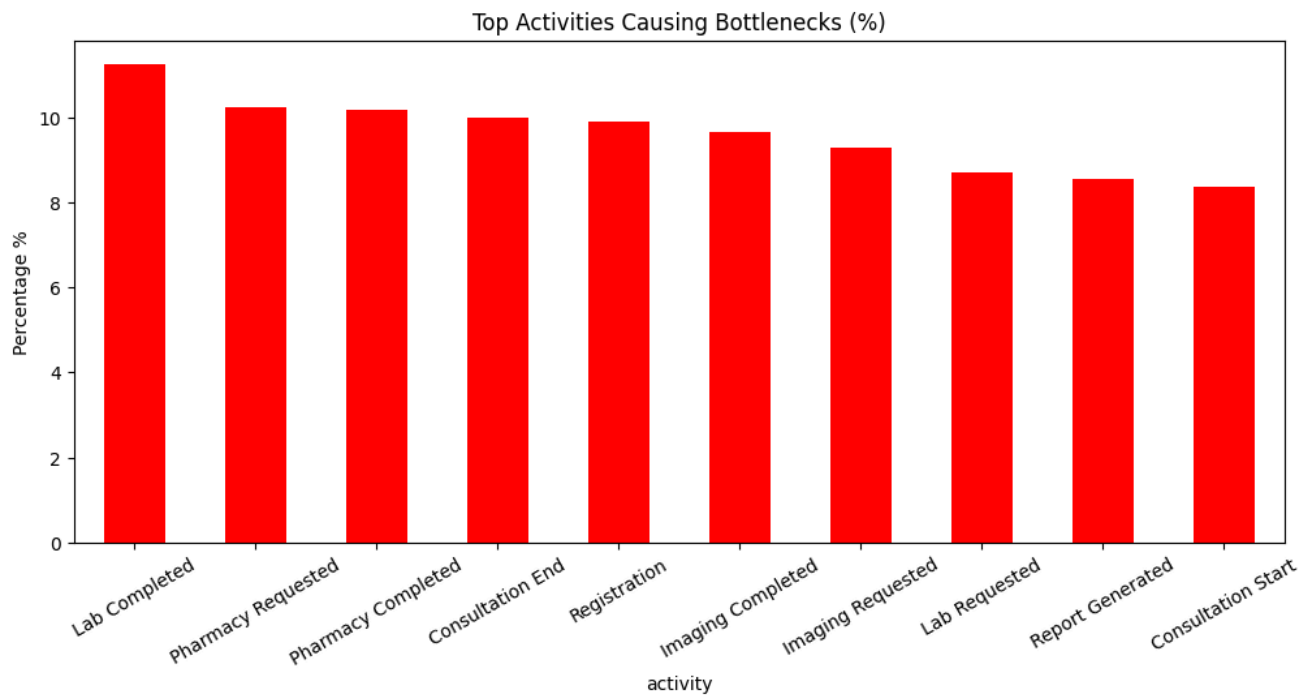
activity	bottleneck_label
Lab Completed	11.242604
Pharmacy Requested	10.230692
Pharmacy Completed	10.172940
Consultation End	10.000000
Registration	9.900990
Imaging Completed	9.649123
Imaging Requested	9.272919
Lab Requested	8.708709
Report Generated	8.557214
Consultation Start	8.373905

dtype: float64

```

plt.figure(figsize=(12,5))
activity_bottleneck.head(10).plot(kind='bar', color='red')
plt.title('Top Activities Causing Bottlenecks (%)')
plt.ylabel('Percentage %')
plt.xticks(rotation=30)
plt.show()

```



```

waiting_by_activity = (
    df.groupby('activity')['waiting_time_minutes']
    .mean()
    .sort_values(ascending=False)
)

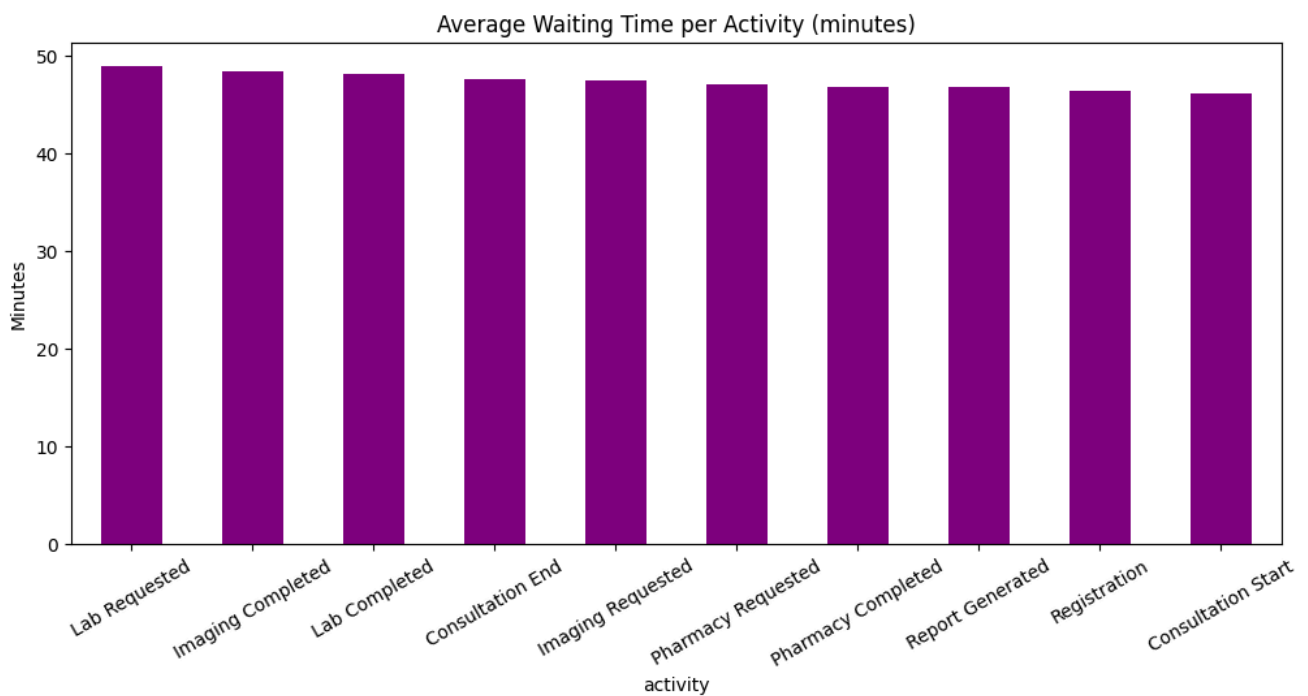
```

```
waiting_by_activity.head(10)
```

waiting_time_minutes	
activity	
Lab Requested	48.967968
Imaging Completed	48.509747
Lab Completed	48.157791
Consultation End	47.634343
Imaging Requested	47.464700
Pharmacy Requested	47.068205
Pharmacy Completed	46.891150
Report Generated	46.821891
Registration	46.512871
Consultation Start	46.133398

```
dtype: float64
```

```
plt.figure(figsize=(12,5))
waiting_by_activity.head(10).plot(kind='bar', color='purple')
plt.title('Average Waiting Time per Activity (minutes)')
plt.ylabel('Minutes')
plt.xticks(rotation=30)
plt.show()
```



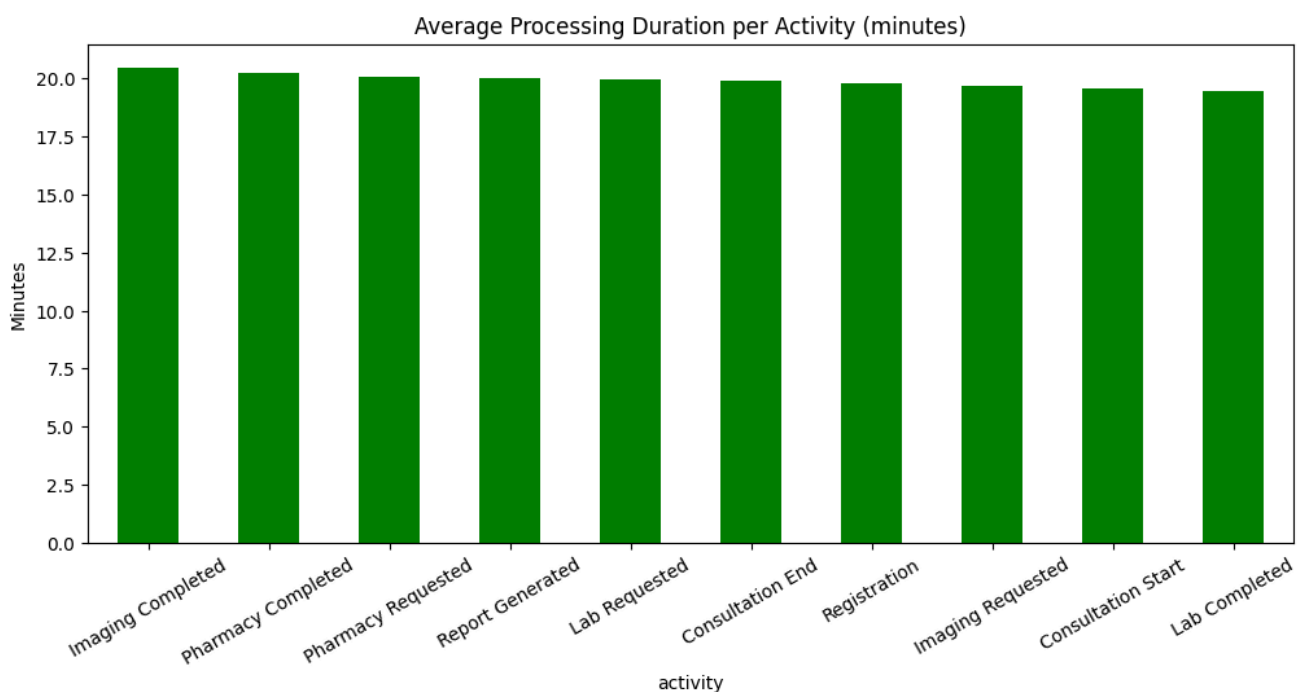
```
processing_by_activity = (
    df.groupby('activity')['processing_duration_minutes']
      .mean()
      .sort_values(ascending=False)
)

processing_by_activity.head(10)
```

activity	processing_duration_minutes
Imaging Completed	20.454191
Pharmacy Completed	20.258393
Pharmacy Requested	20.081244
Report Generated	20.010945
Lab Requested	19.950951
Consultation End	19.903030
Registration	19.786139
Imaging Requested	19.702845
Consultation Start	19.548199
Lab Completed	19.448718

dtype: float64

```
plt.figure(figsize=(12,5))
processing_by_activity.head(10).plot(kind='bar', color='green')
plt.title('Average Processing Duration per Activity (minutes)')
plt.ylabel('Minutes')
plt.xticks(rotation=30)
plt.show()
```



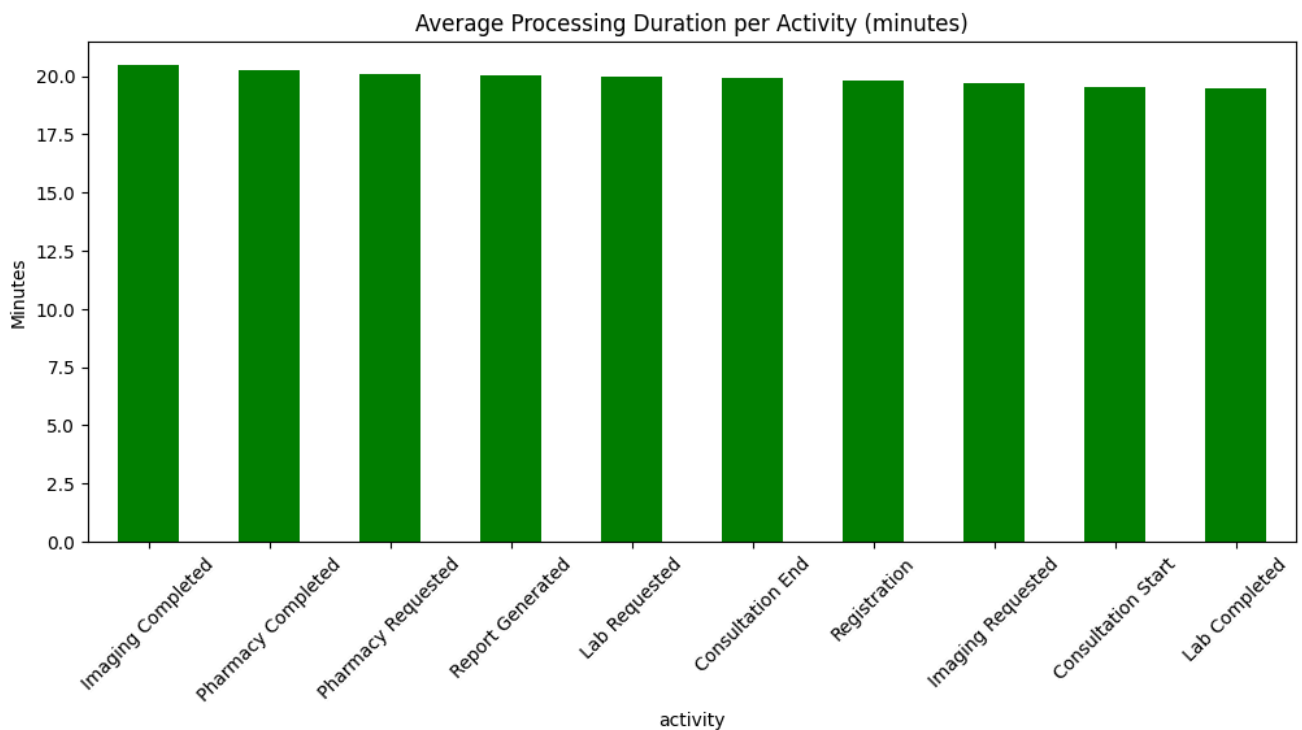
```
processing_by_activity = (
    df.groupby('activity')['processing_duration_minutes']
      .mean()
      .sort_values(ascending=False)
)

processing_by_activity.head(10)
```

activity	processing_duration_minutes
Imaging Completed	20.454191
Pharmacy Completed	20.258393
Pharmacy Requested	20.081244
Report Generated	20.010945
Lab Requested	19.950951
Consultation End	19.903030
Registration	19.786139
Imaging Requested	19.702845
Consultation Start	19.548199
Lab Completed	19.448718

dtype: float64

```
plt.figure(figsize=(12,5))
processing_by_activity.head(10).plot(kind='bar', color='green')
plt.title('Average Processing Duration per Activity (minutes)')
plt.ylabel('Minutes')
plt.xticks(rotation=30)
plt.show()
```



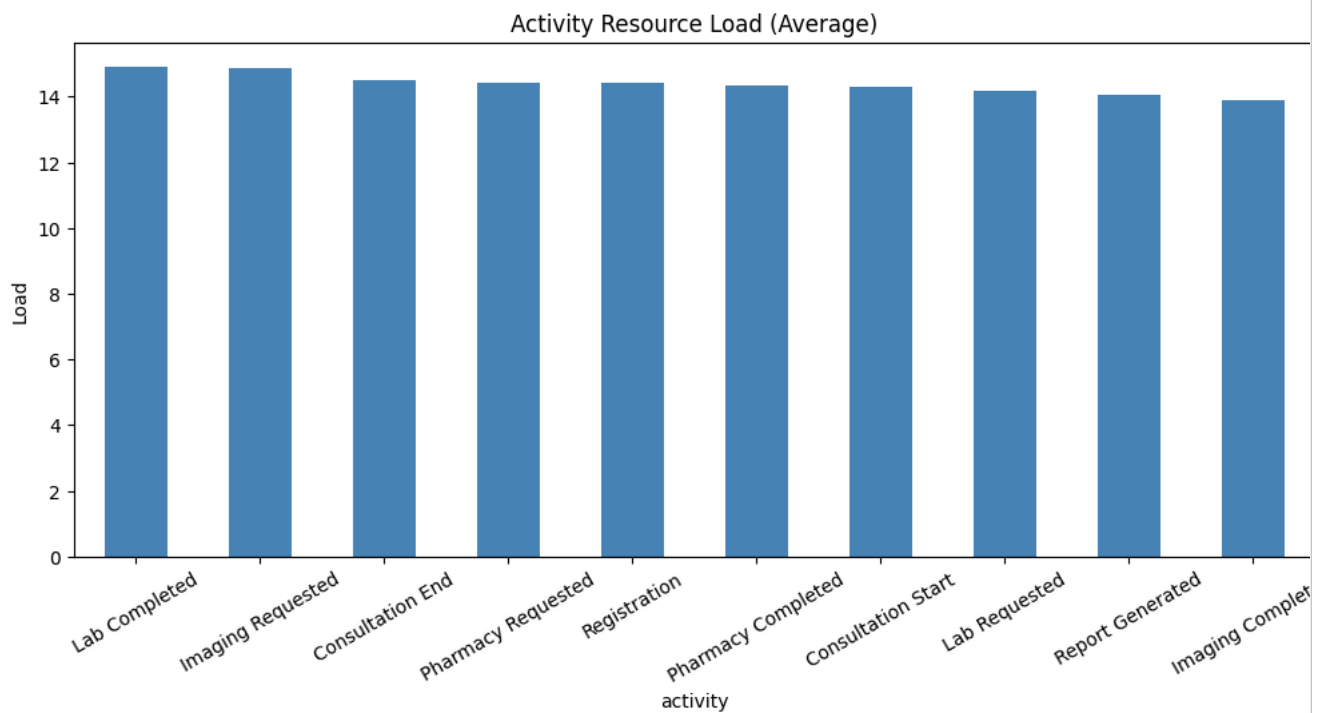
```
resource_load = (
    df.groupby('activity')['activity_resource_load']
    .mean()
    .sort_values(ascending=False)
)

resource_load.head(10)
```

activity_resource_load	
activity	
Lab Completed	14.906193
Imaging Requested	14.854015
Consultation End	14.496222
Pharmacy Requested	14.429067
Registration	14.421733
Pharmacy Completed	14.355239
Consultation Start	14.306641
Lab Requested	14.178799
Report Generated	14.072537
Imaging Completed	13.889337

dtype: float64

```
plt.figure(figsize=(12,5))
resource_load.head(10).plot(kind='bar', color='steelblue')
plt.title('Activity Resource Load (Average)')
plt.ylabel('Load')
plt.xticks(rotation=30)
plt.show()
```



```
doctor_load = (
    df.groupby('doctor_name')['doctor_case_load']
    .mean()
    .sort_values(ascending=False)
)

doctor_load.head(10)
```

doctor_case_load	
doctor_name	
Dr Hassan	13.514079
Dr Fatima	13.171692
Dr Ayesha	13.040541
Dr Bilal	13.039409
Dr Ahmed	12.940286
Dr Khan	12.829607

```
plt.figure(figsize=(12,5))
doctor_load.head(10).plot(kind='bar', color='brown')
plt.title('Doctor Case Load (Average)')
plt.ylabel('Cases')
plt.xticks(rotation=30)
plt.show()
```

